



INSIGHTSMASTERY ACADEMY

# DAX MASTERY STUDENT HANDBOOK

FINAL EDITION

---

SUMX • COUNTX • CALCULATE — Complete Deep Dive  
From Absolute Beginner to Advanced

SUM • SUMX • COUNTX • AVERAGEX • RANKX

CALCULATE — 20 Versions, Every Filter Pattern

FILTER • ALL • REMOVEFILTERS • ALLEXCEPT • ALLSELECTED • KEEPFILTERS  
USERELATIONSHIP • CROSSFILTER • CALCULATETABLE • CONTEXT TRANSITION

Includes Classroom Notes — Class of 29 September 2026

Professional Training Edition

Designed for Classroom & Self-Learning

[www.insightsmastry.in](http://www.insightsmastry.in)



# Contents

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>1. How to Use This Handbook</b>                                     | <b>4</b>  |
| <b>2. Our Standard Sample Dataset</b>                                  | <b>6</b>  |
| <b>3. DAX Foundations in Brief</b>                                     | <b>8</b>  |
| 3.1 What is DAX?                                                       | 8         |
| 3.2 Measures vs Calculated Columns                                     | 8         |
| 3.3 Filter Context                                                     | 8         |
| 3.4 Row Context                                                        | 9         |
| 3.5 The Aggregation Family at a Glance                                 | 9         |
| <b>4. SUMX — Deep Dive</b>                                             | <b>10</b> |
| 4.1 SUMX Step by Step                                                  | 10        |
| 4.2 Pattern 1 — SUMX over one column (same as SUM)                     | 11        |
| 4.3 Pattern 2 — Quantity × Unit Price                                  | 11        |
| 4.4 Pattern 3 — Row-level subtraction (Profit)                         | 12        |
| 4.5 Pattern 4 — SUMX with FILTER, one condition                        | 12        |
| 4.6 Pattern 5 — SUMX with FILTER, AND (&&)                             | 12        |
| 4.7 Pattern 6 — SUMX with FILTER, OR (  ) and IN                       | 13        |
| 4.8 Pattern 7 — IF logic inside the expression (commission)            | 13        |
| 4.9 Pattern 8 — Tax, discount and constants                            | 14        |
| 4.10 Pattern 9 — SUMX over VALUES (context transition)                 | 14        |
| 4.11 Pattern 10 — SUMX inside CALCULATE                                | 15        |
| 4.12 Related iterators: AVERAGEX, MAXX, MINX, RANKX                    | 15        |
| <b>5. COUNTX — Deep Dive</b>                                           | <b>17</b> |
| 5.1 COUNTX Step by Step                                                | 17        |
| 5.2 Pattern 1 — Basic COUNTX                                           | 17        |
| 5.3 Pattern 2 — Conditional count with IF                              | 17        |
| 5.4 Pattern 3 — COUNTX with FILTER                                     | 18        |
| 5.5 Pattern 4 — Multiple conditions                                    | 18        |
| 5.6 Pattern 5 — Count rows meeting a calculated test                   | 18        |
| 5.7 Pattern 6 — Count customers, not rows (with a measure)             | 19        |
| 5.8 COUNTX vs COUNTROWS vs DISTINCTCOUNT                               | 19        |
| 5.9 CALCULATE + COUNT family                                           | 19        |
| <b>6. CALCULATE — The Heart of DAX</b>                                 | <b>21</b> |
| 6.1 What CALCULATE Does                                                | 21        |
| 6.2 Version 1 — One Boolean filter                                     | 22        |
| 6.3 Version 2 — Multiple filters (AND)                                 | 22        |
| 6.4 Version 3 — OR on the same column (   and IN)                      | 22        |
| 6.5 Version 4 — OR across different columns                            | 23        |
| 6.6 Version 5 — Numeric comparisons                                    | 23        |
| 6.7 Version 6 — Date filters                                           | 23        |
| 6.8 Version 7 — Boolean filter vs FILTER(Table) vs FILTER(ALL(Column)) | 24        |
| 6.9 Version 8 — Filtering by a measure                                 | 24        |
| 6.10 Version 9 — ALL / REMOVEFILTERS — percentage of total             | 25        |



|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| 6.11 Version 10 — ALLEXCEPT — keep only chosen filters .....               | 26        |
| 6.12 Version 11 — ALLSELECTED — percentage of what the user selected ..... | 26        |
| 6.13 Version 12 — KEEPFILTERS — intersect instead of replace .....         | 27        |
| 6.14 Version 13 — Nested CALCULATE — the inner filter wins .....           | 27        |
| 6.15 Version 14 — Overriding one filter while keeping others .....         | 27        |
| 6.16 Version 15 — CALCULATE with iterators .....                           | 28        |
| 6.17 Version 16 — CALCULATE with no filters — context transition .....     | 28        |
| 6.18 Version 17 — Variables with CALCULATE .....                           | 29        |
| 6.19 Version 18 — Running total .....                                      | 29        |
| 6.20 Version 19 — USERRELATIONSHIP and CROSSFILTER .....                   | 30        |
| 6.21 Version 20 — CALCULATETABLE — the table version .....                 | 30        |
| 6.22 All 20 Versions at a Glance .....                                     | 31        |
| <b>7. Classroom Notes — Class of 29 September 2026 .....</b>               | <b>32</b> |
| 7.1 Syntax covered in class .....                                          | 32        |
| 7.2 DAX 1 — SUM .....                                                      | 32        |
| 7.3 DAX 2 — SUMX .....                                                     | 32        |
| 7.4 DAX 3 — SUMX + FILTER, one condition .....                             | 32        |
| 7.5 DAX 4 — SUMX + FILTER, two conditions .....                            | 33        |
| 7.6 DAX 5 — CALCULATE with a date filter .....                             | 33        |
| 7.7 Class Conclusion .....                                                 | 34        |
| <b>8. Common Mistakes and Debugging .....</b>                              | <b>35</b> |
| 8.1 Common Mistakes .....                                                  | 35        |
| 8.2 DAX Debugging Process .....                                            | 35        |
| <b>9. Practice Exercises .....</b>                                         | <b>36</b> |
| 9.1 Level 1 — SUMX .....                                                   | 36        |
| 9.2 Level 2 — COUNTX .....                                                 | 36        |
| 9.3 Level 3 — CALCULATE .....                                              | 36        |
| 9.4 Level 4 — Advanced .....                                               | 36        |
| <b>10. Answer Key .....</b>                                                | <b>37</b> |
| 10.1 Level 1 — SUMX .....                                                  | 37        |
| 10.2 Level 2 — COUNTX .....                                                | 37        |
| 10.3 Level 3 — CALCULATE .....                                             | 38        |
| 10.4 Level 4 — Advanced .....                                              | 38        |
| <b>11. DAX Cheat Sheet .....</b>                                           | <b>39</b> |
| 11.1 Sources and Further Reading .....                                     | 39        |



# 1. How to Use This Handbook

This handbook is written as if an experienced teacher is sitting beside you and explaining every concept slowly. You do not need any programming background. We start from absolute zero and finish with the patterns professionals use every day.

## What is new in this Final Edition

- **SUMX Deep Dive** — 10 patterns, every one worked out on the sample data with its exact result.
- **COUNTX Deep Dive** — conditional counts, counting customers, and COUNTX vs COUNTROWS.
- **CALCULATE — All Versions** — 20 numbered versions, from a single filter to USERELATIONSHIP, CROSSFILTER and running totals.
- **Classroom Notes, 29 September 2026** — the five DAX measures from class, explained and rewritten.
- A **UnitPrice** column added to the sample dataset so that Quantity × UnitPrice examples give real, checkable answers.
- Expected results for every practice exercise.

## Recommended Learning Path

1. Read each chapter in order. Do not skip the fundamentals.
2. Type every DAX formula yourself in Power BI Desktop.
3. Compare your result with the green **Result** line under each example.
4. After each example, change the numbers and observe what happens.
5. Complete the practice exercises at the end, then check the answer key.
6. Use the Cheat Sheet and the CALCULATE Versions table for quick revision.

## Teaching Pattern Used Throughout

For every important function we follow this exact pattern:

*WHAT → WHY → SYNTAX → EVERY WORD → SIMPLE EXAMPLE → RESULT → WHY RESULT → REAL-WORLD EXAMPLE → COMMON MISTAKE → PRACTICE*



## DAX Roadmap

| Stage               | Focus                | Key Concepts                                            |
|---------------------|----------------------|---------------------------------------------------------|
| 1. Foundations      | What is DAX?         | Tables, Columns, Rows, Measures, Calculated Columns     |
| 2. Context Basics   | How DAX sees data    | Filter Context, Row Context                             |
| 3. Aggregation      | Simple totals        | COUNT, COUNTA, COUNTROWS, SUM, AVERAGE                  |
| 4. Iterators        | Row-by-row work      | SUMX, COUNTX, AVERAGEX, MAXX, RANKX                     |
| 5. CALCULATE        | Changing context     | Boolean filters, FILTER, AND / OR, dates, measures      |
| 6. Filter Modifiers | Precise control      | ALL, REMOVEFILTERS, ALLEXCEPT, ALLSELECTED, KEEPFILTERS |
| 7. Relationships    | Model-level control  | USERELATIONSHIP, CROSSFILTER                            |
| 8. Deep Concepts    | How the engine works | Context Transition, Nested CALCULATE, CALCULATETABLE    |
| 9. Mastery          | Real projects        | Class notes, Debugging, Practice, Dashboard             |



## 2. Our Standard Sample Dataset

Throughout this handbook we use one consistent **Sales** table. Memorize its structure. Every result printed in green is calculated from these 10 rows, so you can check your own work.

| Sales ID | Date      | Customer | Product  | Category    | Region | Quantity | Unit Price | Sales Amount | Cost  | Profit | Sales person |
|----------|-----------|----------|----------|-------------|--------|----------|------------|--------------|-------|--------|--------------|
| 1        | 01-Jan-26 | Amit     | Laptop   | Electronics | North  | 2        | 50000      | 100000       | 80000 | 20000  | Ravi         |
| 2        | 02-Jan-26 | Neha     | Mouse    | Electronics | South  | 5        | 1000       | 5000         | 3000  | 2000   | Priya        |
| 3        | 03-Jan-26 | Rahul    | Keyboard | Electronics | North  | 3        | 2000       | 6000         | 3500  | 2500   | Ravi         |
| 4        | 04-Jan-26 | Amit     | Monitor  | Electronics | West   | 2        | 15000      | 30000        | 22000 | 8000   | Amit         |
| 5        | 05-Jan-26 | Priya    | Chair    | Furniture   | South  | 4        | 5000       | 20000        | 14000 | 6000   | Priya        |
| 6        | 06-Jan-26 | Rahul    | Desk     | Furniture   | North  | 1        | 15000      | 15000        | 10000 | 5000   | Ravi         |
| 7        | 07-Jan-26 | Neha     | Laptop   | Electronics | South  | 1        | 50000      | 50000        | 40000 | 10000  | Priya        |
| 8        | 08-Jan-26 | Amit     | Chair    | Furniture   | West   | 2        | 5000       | 10000        | 7000  | 3000   | Amit         |
| 9        | 09-Jan-26 | Rahul    | Mouse    | Electronics | North  | 10       | 1000       | 10000        | 6000  | 4000   | Ravi         |
| 10       | 10-Jan-26 | Priya    | Desk     | Furniture   | South  | 2        | 15000      | 30000        | 20000 | 10000  | Priya        |

**NEW** **UnitPrice** is new in this edition. For every row,  $\text{Quantity} \times \text{UnitPrice} = \text{SalesAmount}$ . Column names used in formulas: `Sales[SalesID]`, `Sales[Date]`, `Sales[Customer]`, `Sales[Product]`, `Sales[Category]`, `Sales[Region]`, `Sales[Quantity]`, `Sales[UnitPrice]`, `Sales[SalesAmount]`, `Sales[Cost]`, `Sales[Profit]`, `Sales[Salesperson]`.

### Quick Facts from the Sample Data

| Measure            | Value                        | Measure                                 | Value                  |
|--------------------|------------------------------|-----------------------------------------|------------------------|
| Total rows         | 10                           | Total Quantity                          | 32                     |
| Total SalesAmount  | 276,000                      | Total Cost                              | 205,500                |
| Total Profit       | 70,500                       | Profit margin ( $70,500 \div 276,000$ ) | 25.54%                 |
| Distinct customers | 4 (Amit, Neha, Priya, Rahul) | Distinct regions                        | 3 (North, South, West) |

### Totals you will see again and again

| Region                  | Orders    | Sales          | Share of total |
|-------------------------|-----------|----------------|----------------|
| North (IDs 1, 3, 6, 9)  | 4         | 131,000        | 47.46%         |
| South (IDs 2, 5, 7, 10) | 4         | 105,000        | 38.04%         |
| West (IDs 4, 8)         | 2         | 40,000         | 14.49%         |
| <b>Total</b>            | <b>10</b> | <b>276,000</b> | <b>100%</b>    |



| Category    | Orders | Sales   | Customer | Sales   |
|-------------|--------|---------|----------|---------|
| Electronics | 6      | 201,000 | Amit     | 140,000 |
| Furniture   | 4      | 75,000  | Neha     | 55,000  |
|             |        |         | Priya    | 50,000  |
|             |        |         | Rahul    | 31,000  |



## 3. DAX Foundations in Brief

### 3.1 What is DAX?

**DAX** stands for **Data Analysis Expressions**. It is the formula language of Power BI, Excel Power Pivot and Analysis Services (SSAS Tabular and Azure). DAX is not a programming language like Python — it is a calculation language designed for business intelligence.

#### Simple Analogy

In Excel you write `=SUM(A1:A10)`. In Power BI you write similar ideas, but DAX understands tables, relationships and filters automatically.

### 3.2 Measures vs Calculated Columns

| Aspect          | Calculated Column                                      | Measure                                            |
|-----------------|--------------------------------------------------------|----------------------------------------------------|
| When calculated | At data refresh, for every row                         | At query time, based on filters                    |
| Context         | Row context                                            | Filter context                                     |
| Storage         | Stored in the model (uses memory)                      | Not stored; calculated on demand                   |
| Typical use     | New attributes, flags, per-row values                  | Totals, averages, percentages, KPIs                |
| Example         | <code>Profit = Sales[SalesAmount] - Sales[Cost]</code> | <code>Total Sales = SUM(Sales[SalesAmount])</code> |

#### Teacher Tip

If the number must change when the user clicks a slicer → use a **Measure**. If it is a fixed attribute of each row → a **Calculated Column** is often fine.

### 3.3 Filter Context

Filter context means **"the filters currently affecting your calculation."** It comes from slicers, page and visual filters, the rows and columns of a matrix, cross-filtering, relationships, and filters written inside CALCULATE.

```
Total Sales =  
SUM(Sales[SalesAmount])
```

With no filters this returns **276,000**. Put Region on the rows of a matrix and the same measure returns North 131,000, South 105,000, West 40,000. You never write three measures — one measure plus the visual's filter context does the work.

#### Window Analogy

Filter context is a window. You only see the rows the current filters allow. Change a slicer and the window changes — measures recalculate automatically.



## 3.4 Row Context

Row context means Power BI knows **which single row** it is looking at. It exists in calculated columns (always) and inside iterator functions: SUMX, COUNTX, AVERAGEX, FILTER, RANKX and others.

| Row | SalesAmount | Cost  | Calculation    | Profit |
|-----|-------------|-------|----------------|--------|
| 1   | 100000      | 80000 | 100000 – 80000 | 20000  |
| 2   | 5000        | 3000  | 5000 – 3000    | 2000   |
| 3   | 6000        | 3500  | 6000 – 3500    | 2500   |
| ... | ...         | ...   | ...            | ...    |

### Key Distinction

**Row context** = "I am looking at one specific row right now."

**Filter context** = "These are the rows currently visible because of filters."

They are different. CALCULATE can turn a row context into a filter context — this is **context transition** (Chapter 6, Version 16).

## 3.5 The Aggregation Family at a Glance

| Function      | What it does                                    | Example                        | Result  |
|---------------|-------------------------------------------------|--------------------------------|---------|
| COUNT         | Counts numeric (non-blank) values in one column | COUNT(Sales[SalesID])          | 10      |
| COUNTA        | Counts non-blank values, text or number         | COUNTA(Sales[Customer])        | 10      |
| COUNTROWS     | Counts rows in a table                          | COUNTROWS(Sales)               | 10      |
| DISTINCTCOUNT | Counts unique values                            | DISTINCTCOUNT(Sales[Customer]) | 4       |
| SUM           | Adds one numeric column                         | SUM(Sales[SalesAmount])        | 276,000 |
| AVERAGE       | Mean of one numeric column                      | AVERAGE(Sales[SalesAmount])    | 27,600  |



## 4. SUMX — Deep Dive

This is one of the most important functions in DAX. SUM adds an existing column. **SUMX calculates something for every row and then adds the results.**

### 4.1 SUMX Step by Step

#### 1. What is it?

SUMX is an **iterator**. It walks through a table one row at a time, evaluates an expression on each row, then returns the sum of those results.

#### 2. Why do we need it?

SUM accepts only a single column. The moment you need  $\text{Quantity} \times \text{Price}$ ,  $\text{SalesAmount} - \text{Cost}$ , a discount, or IF logic per row, you need SUMX.

#### 3. Syntax

```
SUMX(  
    <table>,  
    <expression>  
)
```

#### 4. Every word

| Part         | Meaning                                                                              |
|--------------|--------------------------------------------------------------------------------------|
| SUMX         | The function name. The X means it iterates (goes row by row).                        |
| <table>      | The rows to walk through. Can be a table name, FILTER(...), VALUES(...) or ALL(...). |
| <expression> | What to calculate on each row. It is evaluated in row context.                       |
| Return value | The sum of all non-blank results.                                                    |

#### 5. How SUMX works internally

```
Take the table (after current filters)  
    ↓  
Row 1 → evaluate expression → keep result  
Row 2 → evaluate expression → keep result  
...   (every visible row)  
    ↓  
Add all the kept results  
    ↓  
Return the total
```



### When to choose SUM vs SUMX

- Use **SUM** when you only need to add an existing numeric column.
- Use **SUMX** when you need to calculate an expression (multiply, subtract, IF logic, etc.) for each row, or total only certain rows.

## 4.2 Pattern 1 — SUMX over one column (same as SUM)

```
Total Sales SUMX =  
SUMX(  
    Sales,  
    Sales[SalesAmount]  
)
```

**Result = 276,000**

The result is identical to `SUM(Sales[SalesAmount])`. It works, but SUM is simpler — prefer SUM here. This pattern exists to teach the shape of SUMX.

## 4.3 Pattern 2 — Quantity × Unit Price

```
Revenue =  
SUMX(  
    Sales,  
    Sales[Quantity] * Sales[UnitPrice]  
)
```

*Read it like English:*

«Go through every row of Sales. For each row multiply Quantity by UnitPrice. Then add all those results together.»

| Row | Quantity | UnitPrice | Calculation        | Row result |
|-----|----------|-----------|--------------------|------------|
| 1   | 2        | 50,000    | 2 × 50,000         | 100,000    |
| 2   | 5        | 1,000     | 5 × 1,000          | 5,000      |
| 3   | 3        | 2,000     | 3 × 2,000          | 6,000      |
| 9   | 10       | 1,000     | 10 × 1,000         | 10,000     |
| ... | ...      | ...       | ...                | ...        |
|     |          |           | Sum of all 10 rows | 276,000    |

**Result = 276,000**

Why can't we write `SUM(Sales[Quantity] * Sales[UnitPrice])`? Because SUM only accepts a column, not an expression. SUMX exists exactly for this reason. Note: `SUM(Sales[Quantity]) * SUM(Sales[UnitPrice])` gives  $32 \times 159,000 = 5,088,000$  — completely wrong. Multiply first, then add.



## 4.4 Pattern 3 — Row-level subtraction (Profit)

```
Total Profit X =  
SUMX(  
    Sales,  
    Sales[SalesAmount] - Sales[Cost]  
)
```

**Result = 70,500**

No Profit column needed: the subtraction happens row by row inside the iterator, so the model does not have to store an extra calculated column.

## 4.5 Pattern 4 — SUMX with FILTER, one condition

```
North Sales X =  
SUMX(  
    FILTER( Sales, Sales[Region] = "North" ),  
    Sales[SalesAmount]  
)
```

*Read it like English:*

«First keep only the North rows. Then walk through those rows and add SalesAmount.»

**Result = 131,000 (rows 1, 3, 6, 9 → 100,000 + 6,000 + 15,000 + 10,000)**

## 4.6 Pattern 5 — SUMX with FILTER, AND (&&)

```
North Electronics X =  
SUMX(  
    FILTER(  
        Sales,  
        Sales[Region] = "North"  
        && Sales[Category] = "Electronics"  
    ),  
    Sales[SalesAmount]  
)
```

**Result = 116,000 (rows 1, 3, 9)**



## 4.7 Pattern 6 — SUMX with FILTER, OR (||) and IN

```
North or Furniture X =  
SUMX(  
    FILTER(  
        Sales,  
        Sales[Region] = "North"  
        || Sales[Category] = "Furniture"  
    ),  
    Sales[SalesAmount]  
)
```

**Result = 191,000 (North rows 1, 3, 6, 9 plus Furniture rows 5, 8, 10)**

```
Laptop or Desk X =  
SUMX(  
    FILTER( Sales, Sales[Product] IN { "Laptop", "Desk" } ),  
    Sales[SalesAmount]  
)
```

**Result = 195,000 (Laptop 150,000 + Desk 45,000)**

## 4.8 Pattern 7 — IF logic inside the expression (commission)

Business rule: salespeople earn **5%** commission on orders above 30,000 and **2%** on everything else.

```
Commission =  
SUMX(  
    Sales,  
    IF(  
        Sales[SalesAmount] > 30000,  
        Sales[SalesAmount] * 0.05,  
        Sales[SalesAmount] * 0.02  
    )  
)
```

| Group           | Rows        | Sales          | Rate | Commission    |
|-----------------|-------------|----------------|------|---------------|
| Above 30,000    | 1, 7        | 150,000        | 5%   | 7,500         |
| 30,000 or below | the other 8 | 126,000        | 2%   | 2,520         |
| <b>Total</b>    |             | <b>276,000</b> |      | <b>10,020</b> |

**Result = 10,020**

A single rate on the total (276,000 × 5%) would give 13,800 — wrong. The rate must be decided **per row**, which is exactly what SUMX does.



## 4.9 Pattern 8 — Tax, discount and constants

```
Sales incl. GST =  
SUMX(  
    Sales,  
    Sales[SalesAmount] * 1.18  
)
```

**Result = 325,680**

For a flat rate this equals `SUM(Sales[SalesAmount]) * 1.18`. SUMX becomes essential when the rate differs by row — for example a GST rate stored in a column.

## 4.10 Pattern 9 — SUMX over VALUES (context transition)

SUMX can iterate a list of **unique values** instead of the fact table. When a measure is used inside, each value becomes a filter (context transition).

```
Sales from Big Customers =  
SUMX(  
    VALUES( Sales[Customer] ),  
    IF( [Total Sales] > 50000, [Total Sales] )  
)
```

*Read it like English:*

«Take the list of customers. For each customer, calculate that customer's Total Sales. If it is above 50,000, keep it. Add what you kept.»

| Customer | [Total Sales] for that customer | > 50,000?    | Kept           |
|----------|---------------------------------|--------------|----------------|
| Amit     | 140,000                         | Yes          | 140,000        |
| Neha     | 55,000                          | Yes          | 55,000         |
| Priya    | 50,000                          | No           | —              |
| Rahul    | 31,000                          | No           | —              |
|          |                                 | <b>Total</b> | <b>195,000</b> |

**Result = 195,000**

### Why this works

Inside SUMX there is a row context on each customer. **[Total Sales]** is a measure, and every measure is wrapped in an invisible CALCULATE. That CALCULATE turns the current customer into a filter, so [Total Sales] returns only that customer's sales. Without a measure (plain SUM) this would not happen.



## 4.11 Pattern 10 — SUMX inside CALCULATE

```
North Revenue =  
CALCULATE(  
    SUMX( Sales, Sales[Quantity] * Sales[UnitPrice] ),  
    Sales[Region] = "North"  
)
```

```
CALCULATE  
    ↓ changes filter context to North  
    ↓ only North rows remain  
SUMX iterates those North rows  
    ↓ Quantity × UnitPrice per row  
    ↓ sums the results
```

**Result = 131,000**

## 4.12 Related iterators: AVERAGEX, MAXX, MINX, RANKX

### AVERAGEX — average of ratios vs ratio of totals

```
Avg Row Margin % =  
AVERAGEX(  
    Sales,  
    DIVIDE( Sales[Profit], Sales[SalesAmount] )  
)
```

**Result = 31.50%**

```
Overall Margin % =  
DIVIDE(  
    SUM( Sales[Profit] ),  
    SUM( Sales[SalesAmount] )  
)
```

**Result = 25.54%**

#### Which margin is correct?

Both are correct answers to different questions. AVERAGEX gives every order equal weight, so small orders with high margins (Mouse, Keyboard) pull the average up. The ratio of totals weights by money. Finance teams almost always want the ratio of totals.

### MAXX and MINX

```
Biggest Order = MAXX( Sales, Sales[SalesAmount] )          -- 100,000  
Best Customer Sales = MAXX( VALUES(Sales[Customer]), [Total Sales] ) -- 140,000
```



## RANKX — rank customers by sales

```
Customer Rank =  
RANKX(  
    ALL( Sales[Customer] ),  
    [Total Sales],  
    ,  
    DESC  
)
```

| Customer | Total Sales | Customer Rank |
|----------|-------------|---------------|
| Amit     | 140,000     | 1             |
| Neha     | 55,000      | 2             |
| Priya    | 50,000      | 3             |
| Rahul    | 31,000      | 4             |

`ALL(Sales[Customer])` gives the full list to rank against, ignoring the current row's filter. `DESC` puts the highest value at rank 1.



## 5. COUNTX — Deep Dive

Functions ending with X iterate a table row by row. **COUNTX** counts how many rows give a non-blank result for an expression — which makes it the tool for conditional counting.

### 5.1 COUNTX Step by Step

#### Syntax

```
COUNTX(  
    <table>,  
    <expression>  
)
```

| Part         | Meaning                                                             |
|--------------|---------------------------------------------------------------------|
| <table>      | The rows to walk through: a table, FILTER(...), VALUES(...).        |
| <expression> | Evaluated on each row. Rows where it returns BLANK are not counted. |
| Return value | The number of rows with a non-blank result.                         |

#### The golden rule of COUNTX

COUNTX counts **non-blank results**. To skip a row, make the expression return BLANK — usually with an IF that has no 'else' part.

### 5.2 Pattern 1 — Basic COUNTX

```
Order Count X =  
COUNTX(  
    Sales,  
    Sales[SalesID]  
)
```

**Result = 10**

### 5.3 Pattern 2 — Conditional count with IF

```
High Value Orders =  
COUNTX(  
    Sales,  
    IF( Sales[SalesAmount] > 30000, Sales[SalesID] )  
)
```

1. Row 1: 100,000 > 30,000? Yes → return SalesID → counted.
2. Row 2: 5,000 > 30,000? No → IF returns BLANK → not counted.



3. Row 4: 30,000 > 30,000? No (equal is not greater) → not counted.
4. Row 7: 50,000 > 30,000? Yes → counted.
5. Continue to the last row and return the count.

**Result = 2 (rows 1 and 7)**

Change > to >= and the result becomes 4 (rows 1, 4, 7, 10). Boundaries matter.

## 5.4 Pattern 3 — COUNTX with FILTER

```
North Orders X =  
COUNTX(  
    FILTER( Sales, Sales[Region] = "North" ),  
    Sales[SalesID]  
)
```

**Result = 4**

## 5.5 Pattern 4 — Multiple conditions

```
Bulk Electronics Orders =  
COUNTX(  
    FILTER(  
        Sales,  
        Sales[Category] = "Electronics"  
        && Sales[Quantity] >= 3  
    ),  
    Sales[SalesID]  
)
```

**Result = 3 (rows 2, 3, 9 — Mouse × 5, Keyboard × 3, Mouse × 10)**

## 5.6 Pattern 5 — Count rows meeting a calculated test

```
Orders with Profit ≥ 5000 =  
COUNTX(  
    Sales,  
    IF( Sales[SalesAmount] - Sales[Cost] >= 5000, 1 )  
)
```

**Result = 6 (rows 1, 4, 5, 6, 7, 10)**



## 5.7 Pattern 6 — Count customers, not rows (with a measure)

```
Big Customers =  
COUNTX(  
    VALUES( Sales[Customer] ),  
    IF( [Total Sales] > 50000, 1 )  
)
```

**Result = 2 (Amit 140,000 and Neha 55,000)**

Iterating `VALUES(Sales[Customer])` counts **customers**. Iterating `Sales` would count **orders**. Always ask: what am I counting?

## 5.8 COUNTX vs COUNTROWS vs DISTINCTCOUNT

| Formula                                         | Counts                          | Result |
|-------------------------------------------------|---------------------------------|--------|
| COUNTROWS(Sales)                                | Every visible row               | 10     |
| COUNTROWS(FILTER(Sales, Sales[Region]="North")) | Rows passing the test           | 4      |
| COUNTX(Sales, Sales[SalesID])                   | Rows where SalesID is not blank | 10     |
| DISTINCTCOUNT(Sales[Customer])                  | Unique customers                | 4      |
| COUNTROWS(VALUES(Sales[Region]))                | Unique regions visible          | 3      |

### Which one should I use?

- Counting rows with a condition → **COUNTROWS(FILTER(...))** or **CALCULATE(COUNTROWS(Sales), filter)** — clearest.
- Counting non-blank results of a calculation → **COUNTX**.
- Counting unique things → **DISTINCTCOUNT**.

COUNTX and COUNTROWS differ only when the expression can be blank: if SalesID were blank on one row, COUNTX would return 9 and COUNTROWS 10.

## 5.9 CALCULATE + COUNT family

```
North Order Count =  
CALCULATE(  
    COUNTROWS( Sales ),  
    Sales[Region] = "North"  
)
```

**Result = 4**



```
North Customers =  
CALCULATE(  
    DISTINCTCOUNT( Sales[Customer] ),  
    Sales[Region] = "North"  
)
```

**Result = 2 (Amit and Rahul)**



## 6. CALCULATE — The Heart of DAX

If you master CALCULATE, you master DAX. This chapter explains exactly how it works, then walks through **20 versions** of CALCULATE — every filter pattern you will meet in real reports.

### 6.1 What CALCULATE Does

#### 1. What is it?

CALCULATE **evaluates an expression in a modified filter context**. You give it something to calculate, and a list of filters to add, replace or remove first.

#### 2. Syntax

```
CALCULATE(  
    <expression>,  
    <filter1>,  
    <filter2>,  
    ...  
)
```

| Part            | Meaning                                                                                                                                                          |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <expression>    | Always first. Any scalar calculation: a measure, SUM, COUNTROWS, SUMX, AVERAGEX...                                                                               |
| <filter>        | Zero or more. A Boolean condition, a table expression (FILTER, VALUES, ALL...), or a filter modifier (REMOVEFILTERS, KEEPFILTERS, USERELATIONSHIP, CROSSFILTER). |
| Several filters | Combined with AND — every filter must be satisfied.                                                                                                              |

#### 3. The order CALCULATE follows

1. **Evaluate the filter arguments** in the original filter context.
2. **Context transition** — if there is a row context, turn the current row into filters.
3. **Apply modifiers** — ALL / REMOVEFILTERS / USERELATIONSHIP / CROSSFILTER.
4. **Apply the filter arguments** — a filter on a column **replaces** any existing filter on that same column (unless wrapped in KEEPFILTERS, which **intersects** instead).
5. **Evaluate the expression** in the new filter context and return the result.

##### The two rules to remember

**Rule 1 — Replace:** a CALCULATE filter on Region replaces any Region filter coming from a slicer or matrix row. Filters on other columns stay untouched.

**Rule 2 — AND:** several filter arguments work together; all must be true.



## 6.2 Version 1 — One Boolean filter

```
North Sales =  
CALCULATE(  
    [Total Sales],  
    Sales[Region] = "North"  
)
```

*Read it like English:*

«Calculate Total Sales, but only for the rows where Region is North.»

**Result = 131,000**

More of the same shape: Sales[Category] = "Electronics" → 201,000 · Sales[Salesperson] = "Ravi" → 131,000 · Sales[Region] = "South" → 105,000.

## 6.3 Version 2 — Multiple filters (AND)

```
North Electronics Sales =  
CALCULATE(  
    [Total Sales],  
    Sales[Region] = "North",  
    Sales[Category] = "Electronics"  
)
```

**Result = 116,000**

Each comma is an AND. Writing Sales[Region] = "North" && Sales[Category] = "Electronics" as one argument gives the same result.

## 6.4 Version 3 — OR on the same column (|| and IN)

```
North or South Sales =  
CALCULATE(  
    [Total Sales],  
    Sales[Region] = "North" || Sales[Region] = "South"  
)
```

```
North or South Sales (IN) =  
CALCULATE(  
    [Total Sales],  
    Sales[Region] IN { "North", "South" }  
)
```

**Result = 236,000 (131,000 + 105,000)**

IN { ... } is cleaner once you have three or more values. The opposite, NOT Sales[Region] IN { "West" }, also returns 236,000.



## 6.5 Version 4 — OR across different columns

You cannot use commas for OR — commas mean AND. For OR across two columns, filter a table that holds both columns:

```
North or Furniture Sales =  
CALCULATE(  
    [Total Sales],  
    FILTER(  
        ALL( Sales[Region], Sales[Category] ),  
        Sales[Region] = "North" || Sales[Category] = "Furniture"  
    )  
)
```

**Result = 191,000**

ALL(Sales[Region], Sales[Category]) builds the list of every Region–Category combination; FILTER keeps the combinations that satisfy the OR. In current Power BI versions a Boolean filter may reference several columns of the same table, so Sales[Region] = "North" || Sales[Category] = "Furniture" written directly also works.

## 6.6 Version 5 — Numeric comparisons

```
High Value Sales =  
CALCULATE(  
    [Total Sales],  
    Sales[SalesAmount] > 30000  
)
```

**Result = 150,000 (rows 1 and 7)**

```
Mid-range Sales =  
CALCULATE(  
    [Total Sales],  
    Sales[SalesAmount] >= 10000 && Sales[SalesAmount] <= 30000  
)
```

**Result = 115,000 (rows 4, 5, 6, 8, 9, 10)**

## 6.7 Version 6 — Date filters

```
Sales Before 6 Jan =  
CALCULATE(  
    [Total Sales],  
    Sales[Date] < DATE( 2026, 1, 6 )  
)
```

**Result = 161,000 (rows 1–5)**



```
Sales 3-7 Jan =
CALCULATE(
    [Total Sales],
    Sales[Date] >= DATE( 2026, 1, 3 ) && Sales[Date] <= DATE( 2026, 1, 7 )
)
```

**Result = 121,000 (rows 3-7)**

#### Common mistake

`Sales[Date] < "2026-01-06"` compares a date with text. Always build dates with `DATE(year, month, day)`.

## 6.8 Version 7 — Boolean filter vs `FILTER(Table)` vs `FILTER(ALL(Column))`

These three look alike but behave differently inside a visual. A Boolean filter is shorthand for `FILTER(ALL(column), condition)`, so it **replaces** the existing filter on that column. `FILTER(Sales, ...)` iterates only the rows already visible, so it **keeps** the existing filter.

```
A Boolean      = CALCULATE( [Total Sales], Sales[Region] = "North" )
B FILTER(ALL)  = CALCULATE( [Total Sales],
                             FILTER( ALL( Sales[Region] ), Sales[Region] = "North" ) )
C FILTER(table)= CALCULATE( [Total Sales],
                             FILTER( Sales, Sales[Region] = "North" ) )
```

Put Region on the rows of a matrix and add all three:

| Matrix row   | A Boolean      | B FILTER(ALL)  | C FILTER(Sales) |
|--------------|----------------|----------------|-----------------|
| North        | 131,000        | 131,000        | 131,000         |
| South        | 131,000        | 131,000        | (blank)         |
| West         | 131,000        | 131,000        | (blank)         |
| <b>Total</b> | <b>131,000</b> | <b>131,000</b> | <b>131,000</b>  |

#### Best practice

Prefer Boolean filters on columns (A). They are clearer and faster. Use `FILTER(Sales, ...)` only when you need a condition the Boolean form cannot express — and avoid filtering an entire large table when filtering one column is enough.

## 6.9 Version 8 — Filtering by a measure

A Boolean filter cannot contain a measure. `CALCULATE([Total Sales], [Total Sales] > 50000)` returns an error. Iterate a list with `FILTER` instead:



```
Sales from Big Customers =  
CALCULATE(  
    [Total Sales],  
    FILTER(  
        VALUES( Sales[Customer] ),  
        [Total Sales] > 50000  
    )  
)
```

**Result = 195,000 (Amit 140,000 + Neha 55,000)**

FILTER walks the customer list; for each customer [Total Sales] is evaluated (context transition); only Amit and Neha pass; CALCULATE then totals their sales.

## 6.10 Version 9 — ALL / REMOVEFILTERS — percentage of total

```
All Region Sales =  
CALCULATE(  
    [Total Sales],  
    REMOVEFILTERS( Sales[Region] )  
)
```

```
Sales % of Total =  
DIVIDE(  
    [Total Sales],  
    CALCULATE( [Total Sales], REMOVEFILTERS( Sales[Region] ) )  
)
```

| Region       | Total Sales    | All Region Sales | Sales % of Total |
|--------------|----------------|------------------|------------------|
| North        | 131,000        | 276,000          | 47.46%           |
| South        | 105,000        | 276,000          | 38.04%           |
| West         | 40,000         | 276,000          | 14.49%           |
| <b>Total</b> | <b>276,000</b> | <b>276,000</b>   | <b>100%</b>      |

ALL(Sales[Region]) works the same way inside CALCULATE. Microsoft recommends REMOVEFILTERS when your goal is only to remove filters, because the name says what it does. REMOVEFILTERS(Sales) clears every filter on the Sales table; REMOVEFILTERS() with no argument clears all filters in the model.



## 6.11 Version 10 — ALLEXCEPT — keep only chosen filters

```
Category Total =  
CALCULATE(  
    [Total Sales],  
    ALLEXCEPT( Sales, Sales[Category] )  
)
```

*Read it like English:*

«Remove every filter on the Sales table except the Category filter.»

| Matrix: Category / Region | Total Sales | Category Total | Share of category |
|---------------------------|-------------|----------------|-------------------|
| Electronics / North       | 116,000     | 201,000        | 57.71%            |
| Electronics / South       | 55,000      | 201,000        | 27.36%            |
| Electronics / West        | 30,000      | 201,000        | 14.93%            |
| Furniture / North         | 15,000      | 75,000         | 20.00%            |
| Furniture / South         | 50,000      | 75,000         | 66.67%            |
| Furniture / West          | 10,000      | 75,000         | 13.33%            |

Share of category = `DIVIDE([Total Sales], [Category Total])`. `ALLEXCEPT` is ideal for "% within parent" measures.

## 6.12 Version 11 — ALLSELECTED — percentage of what the user selected

```
Sales % of Selected =  
DIVIDE(  
    [Total Sales],  
    CALCULATE( [Total Sales], ALLSELECTED( Sales[Region] ) )  
)
```

A Region slicer has **North** and **South** selected. The matrix shows:

| Region       | Total Sales    | % of Total (ALL) | % of Selected (ALLSELECTED) |
|--------------|----------------|------------------|-----------------------------|
| North        | 131,000        | 47.46%           | 55.51%                      |
| South        | 105,000        | 38.04%           | 44.49%                      |
| <b>Total</b> | <b>236,000</b> | <b>85.51%</b>    | <b>100%</b>                 |

`ALLSELECTED` removes the filter coming from the matrix row but keeps the slicer selection, so the percentages add up to 100% of what is on screen.



## 6.13 Version 12 — KEEPFILTERS — intersect instead of replace

```
North Sales Keep =  
CALCULATE(  
    [Total Sales],  
    KEEPFILTERS( Sales[Region] = "North" )  
)
```

| Matrix row | North Sales (Version 1) | North Sales Keep |
|------------|-------------------------|------------------|
| North      | 131,000                 | 131,000          |
| South      | 131,000                 | (blank)          |
| West       | 131,000                 | (blank)          |
| Total      | 131,000                 | 131,000          |

Without KEEPFILTERS, the North filter **replaces** the South row filter. With KEEPFILTERS, both filters apply: South AND North has no rows, so the result is blank. Use it when a measure must respect what the user is looking at.

## 6.14 Version 13 — Nested CALCULATE — the inner filter wins

```
Inner Wins =  
CALCULATE(  
    CALCULATE( [Total Sales], Sales[Region] = "South" ),  
    Sales[Region] = "North"  
)
```

**Result = 105,000 (South)**

The outer CALCULATE sets Region = North, then the inner CALCULATE replaces it with South. Wrap the inner filter in KEEPFILTERS and the result becomes **blank** (North AND South).

## 6.15 Version 14 — Overriding one filter while keeping others

```
Electronics Sales (any category slicer) =  
CALCULATE(  
    [Total Sales],  
    Sales[Category] = "Electronics"  
)
```

With a slicer set to **Region = South** and **Category = Furniture**, this measure returns **55,000**: the Category slicer is replaced by Electronics, but the Region = South filter is kept (rows 2 and 7).

### Think in columns

CALCULATE only touches the columns named in its filters. Everything else from the report stays in place.



## 6.16 Version 15 — CALCULATE with iterators

```
North Average Profit =  
CALCULATE(  
    AVERAGEX( Sales, Sales[SalesAmount] - Sales[Cost] ),  
    Sales[Region] = "North"  
)
```

**Result = 7,875 (31,500 profit ÷ 4 North orders)**

```
Electronics Bulk Orders =  
CALCULATE(  
    COUNTX( Sales, IF( Sales[Quantity] >= 3, Sales[SalesID] ) ),  
    Sales[Category] = "Electronics"  
)
```

**Result = 3**

## 6.17 Version 16 — CALCULATE with no filters — context transition

In a calculated column there is a row context but no filter context. SUM ignores the row; CALCULATE turns the row into a filter.

```
Col A = SUM( Sales[SalesAmount] )  
Col B = CALCULATE( SUM( Sales[SalesAmount] ) )  
Col C = CALCULATE( SUM( Sales[SalesAmount] ),  
    ALLEXCEPT( Sales, Sales[Customer] ) )
```

| Row | Customer | SalesAmount | Col A   | Col B   | Col C   |
|-----|----------|-------------|---------|---------|---------|
| 1   | Amit     | 100,000     | 276,000 | 100,000 | 140,000 |
| 2   | Neha     | 5,000       | 276,000 | 5,000   | 55,000  |
| 4   | Amit     | 30,000      | 276,000 | 30,000  | 140,000 |
| 7   | Neha     | 50,000      | 276,000 | 50,000  | 55,000  |

- **Col A** — no context transition: every row sees the whole table.
- **Col B** — context transition: the current row becomes a filter, so the row sees only itself.
- **Col C** — context transition, then ALLEXCEPT keeps only the Customer filter: each row shows its customer's total.

```
ROW CONTEXT → CALCULATE (or a measure call) → CONTEXT TRANSITION → FILTER CONTEXT →  
EVALUATION
```

**Remember**

Calling a measure inside an iterator or a calculated column triggers context transition automatically, because every measure is wrapped in an invisible CALCULATE.

## 6.18 Version 17 — Variables with CALCULATE

```
Profit Margin % =  
VAR TotalSales = [Total Sales]  
VAR TotalProfit = CALCULATE( SUM( Sales[Profit] ) )  
RETURN  
    DIVIDE( TotalProfit, TotalSales )
```

**Result = 25.54%**

```
North Share % =  
VAR NorthSales = CALCULATE( [Total Sales], Sales[Region] = "North" )  
VAR AllSales = CALCULATE( [Total Sales], REMOVEFILTERS( Sales ) )  
RETURN  
    DIVIDE( NorthSales, AllSales )
```

**Result = 47.46%**

Variables are evaluated once, where they are defined. They make long CALCULATE formulas readable and easy to debug — return one variable at a time to test it.

## 6.19 Version 18 — Running total

```
Running Total =  
CALCULATE(  
    [Total Sales],  
    FILTER(  
        ALL( Sales[Date] ),  
        Sales[Date] <= MAX( Sales[Date] )  
    )  
)
```

| Date      | Total Sales | Running Total |
|-----------|-------------|---------------|
| 01-Jan-26 | 100,000     | 100,000       |
| 02-Jan-26 | 5,000       | 105,000       |
| 03-Jan-26 | 6,000       | 111,000       |
| 05-Jan-26 | 20,000      | 161,000       |
| 10-Jan-26 | 30,000      | 276,000       |



MAX(Sales[Date]) is read in the original context (the matrix row), then ALL removes the date filter and FILTER keeps every date up to that one.

With a marked Date table, time-intelligence functions are shortcuts for the same idea: `CALCULATE([Total Sales], DATESYTD('Date'[Date]))` for year-to-date and `CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))` for last year.

## 6.20 Version 19 — USERELATIONSHIP and CROSSFILTER

These modifiers change how relationships behave, so they need a model with more than one table. Suppose Sales has **OrderDate** (active relationship to 'Date') and **ShipDate** (inactive relationship).

```
Sales by Ship Date =
CALCULATE(
    [Total Sales],
    USERELATIONSHIP( Sales[ShipDate], 'Date'[Date] )
)
```

USERELATIONSHIP switches on the inactive relationship for this calculation only; the active one is switched off meanwhile.

```
Customers Who Bought =
CALCULATE(
    DISTINCTCOUNT( Customer[CustomerID] ),
    CROSSFILTER( Sales[CustomerID], Customer[CustomerID], BOTH )
)
```

CROSSFILTER changes the filter direction for one calculation (Both, OneWay, or None to disable the relationship) — safer than setting the whole model to bidirectional.

## 6.21 Version 20 — CALCULATETABLE — the table version

CALCULATETABLE does exactly what CALCULATE does, but returns a **table** instead of a single value.

```
North Customer Count =
COUNTROWS(
    CALCULATETABLE(
        VALUES( Sales[Customer] ),
        Sales[Region] = "North"
    )
)
```

**Result = 2 (Amit, Rahul)**



## 6.22 All 20 Versions at a Glance

| #  | Version                | Key filter                                      | Result            |
|----|------------------------|-------------------------------------------------|-------------------|
| 1  | One Boolean filter     | Sales[Region] = "North"                         | 131,000           |
| 2  | Multiple filters (AND) | ..., Region = "North", Category = "Electronics" | 116,000           |
| 3  | OR, same column        | Sales[Region] IN {"North","South"}              | 236,000           |
| 4  | OR, different columns  | FILTER(ALL(Region, Category), ...    ...)       | 191,000           |
| 5  | Numeric comparison     | Sales[SalesAmount] > 30000                      | 150,000           |
| 6  | Date filter            | Sales[Date] < DATE(2026,1,6)                    | 161,000           |
| 7  | Boolean vs FILTER      | FILTER(Sales, ...) keeps existing filters       | —                 |
| 8  | Filter by a measure    | FILTER(VALUES(Customer), [Total Sales] > 50000) | 195,000           |
| 9  | % of total             | REMOVEFILTERS(Sales[Region])                    | 276,000           |
| 10 | Keep one filter        | ALLEXCEPT(Sales, Sales[Category])               | 201,000 / 75,000  |
| 11 | % of selection         | ALLSELECTED(Sales[Region])                      | 236,000           |
| 12 | Intersect              | KEEPFILTERS(Region = "North")                   | blank on South    |
| 13 | Nested                 | inner filter wins                               | 105,000           |
| 14 | Override one column    | other slicers still apply                       | 55,000            |
| 15 | With iterators         | AVERAGEX, COUNTX inside                         | 7,875 / 3         |
| 16 | Context transition     | CALCULATE(SUM(...)) in a column                 | row value         |
| 17 | Variables              | VAR ... RETURN DIVIDE(...)                      | 25.54%            |
| 18 | Running total          | FILTER(ALL(Date), Date <= MAX(Date))            | 276,000 on 10 Jan |
| 19 | Relationships          | USERELATIONSHIP, CROSSFILTER                    | model-based       |
| 20 | Table result           | CALCULATETABLE(VALUES(...), ...)                | 2 rows            |



## 7. Classroom Notes — Class of 29 September 2026

This chapter records the five measures written in class, in order. They use a **Shop** table (a coffee-shop sales log) with the columns `OrderDate`, `Product`, `PaymentMethod` and `SalesAmount`. The class moved from a plain SUM, to conditional totals with SUMX + FILTER, to CALCULATE for stacking several filters.

### 7.1 Syntax covered in class

| Function  | Syntax                                       | Order of arguments |
|-----------|----------------------------------------------|--------------------|
| CALCULATE | CALCULATE(Expression, Filter1, Filter2, ...) | Expression first   |
| SUMX      | SUMX(Table, Expression)                      | Table first        |
| COUNTX    | COUNTX(Table, Expression)                    | Table first        |
| AVERAGEX  | AVERAGEX(Table, Expression)                  | Table first        |
| RANKX     | RANKX(Table, Expression)                     | Table first        |

The table argument of any X function can itself be `FILTER(...)`. That is how you get a conditional total.

### 7.2 DAX 1 — SUM

```
Dax1 =  
-- SUM('Table'[Column Name])  
SUM( 'Shop'[SalesAmount] )
```

A plain total of one column, with no conditions of its own. Slicers and visuals do all the filtering.

### 7.3 DAX 2 — SUMX

```
Dax2 =  
-- SUMX(Table, Expression) – the table can be narrowed with FILTER  
SUMX( 'Shop', 'Shop'[SalesAmount] )
```

Same result as DAX 1, because the expression is a single column. SUMX earns its keep when the expression is a calculation or the table is filtered.

### 7.4 DAX 3 — SUMX + FILTER, one condition

```
Dax2_1_Cash =  
SUMX(  
    FILTER( Shop, Shop[PaymentMethod] = "Cash" ),  
    'Shop'[SalesAmount]  
)
```



FILTER keeps only Cash rows; SUMX adds SalesAmount for those rows.

## 7.5 DAX 4 — SUMX + FILTER, two conditions

```
Dax2_1_Cash_Product =  
SUMX(  
    FILTER(  
        Shop,  
        Shop[PaymentMethod] = "Cash"  
        && Shop[Product] = "Latte"  
    ),  
    'Shop'[SalesAmount]  
)
```

&& means AND: a row must be both Cash and Latte.

## 7.6 DAX 5 — CALCULATE with a date filter

```
Dax2_1_2_Cash_Product =  
CALCULATE(  
    SUMX(  
        FILTER(  
            Shop,  
            Shop[PaymentMethod] = "Cash"  
            && Shop[Product] = "Latte"  
        ),  
        'Shop'[SalesAmount]  
    ),  
    FILTER( Shop, Shop[OrderDate] < DATE( 2024, 4, 1 ) )  
)
```

1. CALCULATE first applies its filter: only orders dated before 1 April 2024.
2. Inside that context, SUMX + FILTER keeps the Cash and Latte rows.
3. SUMX adds SalesAmount for what remains: Cash, Latte, before April 2024.

### Recommended rewrite

```
Cash Latte Sales Before Apr 2024 =  
CALCULATE(  
    SUM( Shop[SalesAmount] ),  
    Shop[PaymentMethod] = "Cash",  
    Shop[Product] = "Latte",  
    Shop[OrderDate] < DATE( 2024, 4, 1 )  
)
```

Same number, easier to read, faster to run. Column filters touch one column each; FILTER(Shop, ...) scans the whole table row by row.



## 7.7 Class Conclusion

| Function  | Filters                  | Row-level expression          | Best for                                         | Class example |
|-----------|--------------------------|-------------------------------|--------------------------------------------------|---------------|
| SUM       | None of its own          | No                            | Plain total of one column                        | DAX 1         |
| SUMX      | Via a FILTER table       | Yes                           | Totals of calculated values, or of filtered rows | DAX 2, 3, 4   |
| CALCULATE | Any number, as arguments | Through the expression inside | Any aggregation under several filters            | DAX 5         |



## 8. Common Mistakes and Debugging

### 8.1 Common Mistakes

| Mistake                                                                        | Why it fails                                               | Fix                                                                    |
|--------------------------------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------|
| <code>SUM(Sales[Quantity] * Sales[UnitPrice])</code>                           | SUM takes one column, not an expression                    | <code>SUMX(Sales, Sales[Quantity] * Sales[UnitPrice])</code>           |
| <code>SUM(Qty) * SUM(Price)</code>                                             | Multiplies totals, not rows (5,088,000 instead of 276,000) | Multiply per row with SUMX                                             |
| <code>SUMX(Sales, Sales[SalesAmount])</code> everywhere                        | Works, but SUM is simpler                                  | Use SUM for one column                                                 |
| <code>SUMX(Sales[SalesAmount], Sales)</code>                                   | Arguments reversed                                         | Table first                                                            |
| <code>CALCULATE(Filter, SUM(...))</code>                                       | Arguments reversed                                         | Expression first                                                       |
| <code>CALCULATE([Total Sales], [Total Sales] &gt; 50000)</code>                | Boolean filters cannot use measures                        | <code>FILTER(VALUES(Sales[Customer]), [Total Sales] &gt; 50000)</code> |
| <code>CALCULATE(..., Region="North", Category="Furniture")</code> expecting OR | Commas mean AND                                            | Version 4: <code>FILTER(ALL(col1, col2), ...    ...)</code>            |
| <code>[PaymentMethod] = Cash</code>                                            | Text values need quotes                                    | <code>= "Cash"</code>                                                  |
| <code>[Date] &lt; "2024-04-01"</code>                                          | Compares a date with text                                  | <code>DATE(2024, 4, 1)</code>                                          |
| <code>FILTER(Sales, Region = "North")</code> for a simple filter               | Scans the whole table                                      | <code>Sales[Region] = "North"</code>                                   |
| COUNTX without IF for a conditional count                                      | Counts every non-blank row                                 | Return BLANK for rows to skip                                          |
| AVERAGEX of ratios as overall margin                                           | Weights every row equally                                  | <code>DIVIDE(SUM(Profit), SUM(Sales))</code>                           |

### 8.2 DAX Debugging Process

1. **Check the table** — does it exist and contain the expected rows?
2. **Check the column** — spelled correctly? Right data type? Numbers stored as text cause problems.
3. **Check filter context** — which slicers, page filters and visual filters are active?
4. **Check row context** — are you inside an iterator or a calculated column?
5. **Test the base measure** — does the simplest version (e.g. SUM) return the right number?
6. **Test FILTER separately** — a temporary measure returning `COUNTROWS(FILTER(...))` shows how many rows pass.
7. **Build CALCULATE one filter at a time** — add a filter, check, add the next.
8. **Use variables** — RETURN each VAR in turn to see its value.

#### Pro Tip

Create temporary measures named Test1, Test2 while debugging and compare them against the green results in this handbook. Delete them once the final measure works.



## 9. Practice Exercises

Use the sample Sales table. Write each measure in Power BI Desktop, then check the expected result in the next chapter.

### 9.1 Level 1 — SUMX

1. Total revenue using  $\text{Quantity} \times \text{UnitPrice}$ .
2. Total profit without using the Profit column.
3. Total sales for the South region using SUMX + FILTER.
4. Total sales of Electronics in the North using SUMX + FILTER with &&.
5. Commission: 5% on orders above 30,000, otherwise 2%.

### 9.2 Level 2 — COUNTX

1. Number of orders with SalesAmount of 30,000 or more.
2. Number of Furniture orders.
3. Number of Electronics orders with Quantity of 3 or more.
4. Number of customers whose Total Sales is above 50,000.

### 9.3 Level 3 — CALCULATE

1. Furniture sales in the South.
2. Sales for Laptop or Desk, using IN.
3. Sales for the North region or the Furniture category.
4. Sales before 6 January 2026.
5. Region share of total, using REMOVEFILTERS.
6. Share of the selected regions, using ALLSELECTED.
7. Region sales that return blank on rows other than North (KEEPFILTERS).

### 9.4 Level 4 — Advanced

1. Sales from customers whose Total Sales is above 50,000 (CALCULATE + FILTER over VALUES).
2. Average profit per order in the North, using CALCULATE + AVERAGEX.
3. Running total by date.
4. Profit margin % using variables.
5. Number of North customers using CALCULATETABLE.



## 10. Answer Key

### 10.1 Level 1 — SUMX

```
1 Revenue      = SUMX( Sales, Sales[Quantity] * Sales[UnitPrice] )
2 Profit X     = SUMX( Sales, Sales[SalesAmount] - Sales[Cost] )
3 South X      = SUMX( FILTER( Sales, Sales[Region] = "South" ), Sales[SalesAmount] )
4 North Elec X = SUMX( FILTER( Sales, Sales[Region] = "North"
                        && Sales[Category] = "Electronics" ), Sales[SalesAmount] )
5 Commission   = SUMX( Sales, IF( Sales[SalesAmount] > 30000,
                        Sales[SalesAmount] * 0.05, Sales[SalesAmount] * 0.02 ) )
```

| # | Expected result |
|---|-----------------|
| 1 | 276,000         |
| 2 | 70,500          |
| 3 | 105,000         |
| 4 | 116,000         |
| 5 | 10,020          |

### 10.2 Level 2 — COUNTX

```
1 COUNTX( Sales, IF( Sales[SalesAmount] >= 30000, Sales[SalesID] ) )
2 COUNTX( FILTER( Sales, Sales[Category] = "Furniture" ), Sales[SalesID] )
3 COUNTX( FILTER( Sales, Sales[Category] = "Electronics"
                && Sales[Quantity] >= 3 ), Sales[SalesID] )
4 COUNTX( VALUES( Sales[Customer] ), IF( [Total Sales] > 50000, 1 ) )
```

| # | Expected result      |
|---|----------------------|
| 1 | 4 (rows 1, 4, 7, 10) |
| 2 | 4                    |
| 3 | 3 (rows 2, 3, 9)     |
| 4 | 2 (Amit, Neha)       |



## 10.3 Level 3 — CALCULATE

```
1 CALCULATE( [Total Sales], Sales[Region] = "South", Sales[Category] = "Furniture" )
2 CALCULATE( [Total Sales], Sales[Product] IN { "Laptop", "Desk" } )
3 CALCULATE( [Total Sales], FILTER( ALL( Sales[Region], Sales[Category] ),
    Sales[Region] = "North" || Sales[Category] = "Furniture" ) )
4 CALCULATE( [Total Sales], Sales[Date] < DATE( 2026, 1, 6 ) )
5 DIVIDE( [Total Sales], CALCULATE( [Total Sales], REMOVEFILTERS( Sales[Region] ) ) )
6 DIVIDE( [Total Sales], CALCULATE( [Total Sales], ALLSELECTED( Sales[Region] ) ) )
7 CALCULATE( [Total Sales], KEEPFILTERS( Sales[Region] = "North" ) )
```

| # | Expected result                             |
|---|---------------------------------------------|
| 1 | 50,000                                      |
| 2 | 195,000                                     |
| 3 | 191,000                                     |
| 4 | 161,000                                     |
| 5 | North 47.46%, South 38.04%, West 14.49%     |
| 6 | With North + South selected: 55.51%, 44.49% |
| 7 | North 131,000; South and West blank         |

## 10.4 Level 4 — Advanced

```
1 CALCULATE( [Total Sales], FILTER( VALUES( Sales[Customer] ), [Total Sales] > 50000 ) )
2 CALCULATE( AVERAGEX( Sales, Sales[SalesAmount] - Sales[Cost] ), Sales[Region] = "North"
)
3 CALCULATE( [Total Sales], FILTER( ALL( Sales[Date] ), Sales[Date] <= MAX( Sales[Date] )
) )
4 VAR S = [Total Sales] VAR P = SUM( Sales[Profit] ) RETURN DIVIDE( P, S )
5 COUNTROWS( CALCULATETABLE( VALUES( Sales[Customer] ), Sales[Region] = "North" ) )
```

| # | Expected result                          |
|---|------------------------------------------|
| 1 | 195,000                                  |
| 2 | 7,875                                    |
| 3 | e.g. 161,000 on 5 Jan; 276,000 on 10 Jan |
| 4 | 25.54%                                   |
| 5 | 2                                        |



# 11. DAX Cheat Sheet

| Function        | Purpose                              | Simple Example                                             |
|-----------------|--------------------------------------|------------------------------------------------------------|
| SUM             | Sum a column                         | SUM(Sales[SalesAmount])                                    |
| SUMX            | Sum an expression row by row         | SUMX(Sales, Sales[Quantity] * Sales[UnitPrice])            |
| COUNTROWS       | Count rows in a table                | COUNTROWS(Sales)                                           |
| COUNTX          | Count non-blank expression results   | COUNTX(Sales, IF(Sales[SalesAmount] > 30000, 1))           |
| DISTINCTCOUNT   | Count unique values                  | DISTINCTCOUNT(Sales[Customer])                             |
| AVERAGEX        | Average an expression                | AVERAGEX(Sales, Sales[SalesAmount] - Sales[Cost])          |
| MAXX / MINX     | Largest / smallest expression result | MAXX(Sales, Sales[SalesAmount])                            |
| RANKX           | Rank items by a measure              | RANKX(ALL(Sales[Customer]), [Total Sales])                 |
| CALCULATE       | Change filter context                | CALCULATE([Total Sales], Sales[Region] = "North")          |
| CALCULATETABLE  | Same, returns a table                | CALCULATETABLE(VALUES(Sales[Customer]), ...)               |
| FILTER          | Return a filtered table              | FILTER(Sales, Sales[SalesAmount] > 30000)                  |
| ALL             | Remove filters / return all values   | ALL(Sales[Region])                                         |
| REMOVEFILTERS   | Remove filters (clearest)            | REMOVEFILTERS(Sales[Region])                               |
| ALLEXCEPT       | Remove all except chosen columns     | ALLEXCEPT(Sales, Sales[Category])                          |
| ALLSELECTED     | Keep outer ( slicer ) filters        | ALLSELECTED(Sales[Region])                                 |
| KEEPFILTERS     | Intersect instead of replace         | KEEPFILTERS(Sales[Region] = "North")                       |
| USERELATIONSHIP | Use an inactive relationship         | USERELATIONSHIP(Sales[ShipDate], 'Date'[Date])             |
| CROSSFILTER     | Change filter direction              | CROSSFILTER(Sales[CustomerID], Customer[CustomerID], BOTH) |
| VALUES          | Distinct visible values              | VALUES(Sales[Region])                                      |
| DIVIDE          | Safe division                        | DIVIDE([Profit], [Total Sales])                            |

## 11.1 Sources and Further Reading

Function behaviour in this handbook follows Microsoft's official DAX reference (accessed 30 September 2026):

- CALCULATE function — [learn.microsoft.com/dax/calculate-function-dax](https://learn.microsoft.com/dax/calculate-function-dax)
- KEEPFILTERS function — [learn.microsoft.com/dax/keepfilters-function-dax](https://learn.microsoft.com/dax/keepfilters-function-dax)
- ALLNOBLANKROW and the ALL family — [learn.microsoft.com/dax/allnoblankrow-function-dax](https://learn.microsoft.com/dax/allnoblankrow-function-dax)
- Filter functions — [learn.microsoft.com/dax/filter-functions-dax](https://learn.microsoft.com/dax/filter-functions-dax)



- Relationship functions (USERRELATIONSHIP, CROSSFILTER) — [learn.microsoft.com/dax/relationship-functions-dax](https://learn.microsoft.com/dax/relationship-functions-dax)
- Training module: Modify DAX filter context in Power BI semantic models — [learn.microsoft.com/training/modules/dax-power-bi-modify-filter/](https://learn.microsoft.com/training/modules/dax-power-bi-modify-filter/)

### Continue Your Journey

Next topics: Time Intelligence (TOTALYTD, SAMEPERIODLASTYEAR, DATEADD), virtual relationships (TREATAS), window functions (INDEX, OFFSET, WINDOW), and performance tuning with DAX Studio. Keep practising with real datasets — the more you write and test, the deeper your understanding becomes.

Congratulations on completing the DAX Mastery Student Handbook.  
You now understand not only what formula to write, but why it works.